

LABORATOR 1-2

CLASE ȘI MOȘTENIRE ÎN LIMBAJUL JAVA

1.1. Obiective specifice

Primul capitol are rolul de a prezenta și familiariza studentul cu noțiuni și elemente de bază ale programării orientate pe obiect în limbajul Java cum ar fi: clase și crearea obiectelor, rolul constructorilor, moștenirea și derivarea claselor, suprascrierea metodelor, ascunderea și încapsularea datelor, clase abstracte cu importanța și utilizarea corectă a acestora.

La sfârșitul acestui capitol, studentul va avea posibilitatea să scrie programe Java în care să utilizeze elementele specifice programării orientate pe obiect menționate mai sus.

1.2. Limbajul Java

Limbajul Java:

- este în totalitate **orientat obiect**
 - o permite **crearea claselor de obiecte și instanțieri ale acestora**
 - o permite **încapsularea informațiilor**
 - o permite **moștenirea variabilelor și a metodelor de la o clasă la alta**
- **nu utilizează moștenirea multiplă**, dar pentru a suplini această lipsă, Java oferă o facilitare mai simplă, numită **interfață**, care permite definirea unui anumit comportament pentru o clasă de obiecte, altul decât cel definit de clasa de bază
- **este distribuit**, având implementate biblioteci pentru lucrul în rețea care ne oferă TCP/IP, URL și încărcarea resurselor din rețea. Aplicațiile Java pot accesa foarte ușor rețeaua, folosindu-se de apelurile către un set standard de clase
- este **robust**, legarea funcțiilor se face în timpul execuției și informațiile de compilare sunt disponibile până în momentul rulării aplicației. Acest mod de lucru face ca sistemul să poată determina în orice moment neconcordanța dintre tipul referit la compilare și cel referit în timpul execuției evitându-se astfel posibile intruziuni răuvoitoare în sistem prin intermediul unor referințe falsificate.
- **nu utilizează pointeri**, eliminându-se astfel una din principalele surse de erori
- **eliberează memoria ocupată de obiecte și tablouri în mod automat**, prin mecanismul de colectare de gunoai, evitându-se astfel încercările de eliberare multiplă a unei zone de memorie
- este un **limbaj cu securitate ridicată**, verificându-se la fiecare încărcare codul prin mecanisme de CRC (**Control Redundant Ciclic**) și prin verificarea operațiilor disponibile pentru fiecare set de obiecte. Robustețea este și ea o trăsătură de securitate. La un al doilea nivel, Java are incorporate facilități de protecție a obiectelor din sistem la scriere și/sau citire. Variabilele protejate într-un obiect Java nu pot fi accesate fără a avea drepturile necesare, verificarea fiind făcută în timpul execuției. În plus, mediul de execuție Java poate fi configurat pentru a proteja rețeaua locală, fișierele și celelalte resurse ale calculatorului pe care rulează o aplicație Java
- **are inclus suportul nativ pentru aplicații care lucrează cu mai multe fire de execuție**, inclusiv primitive de sincronizare între firele de execuție. Acest suport este independent de sistemul de operare, dar poate fi conectat, pentru o performanță mai bună, la facilitățile sistemului dacă acestea există
- este **dinamic**, bibliotecile de clase în Java putând fi reutilizate cu foarte mare ușurință. Cunoscuta problemă a fragilității superclasei este rezolvată mai bine decât în C++. Acolo, dacă o superclasă este modificată, trebuie recompilate toate subclasele acesteia pentru că obiectele au o altă structură în memorie. În Java această problemă este rezolvată prin

legarea târzie variabilelor, doar la execuție. Regăsirea variabilelor se face prin nume și nu printr-un deplasament fix. Dacă superclasa nu a șters o parte dintre vechile variabile și metode, ea va putea fi refolosită fără să fie necesară recompilarea subclaselor acesteia. Se elimină astfel necesitatea actualizării aplicațiilor

- lucrează în mod nativ folosind **setul de caractere Unicode**. Acesta este un standard internațional care înlocuiește vechiul set de caractere ASCII. Motivul acestei înlocuiri a fost necesitatea de a reprezenta mai mult de 256 de caractere. Setul de caractere Unicode, fiind reprezentat pe 16 biți are posibilități mult mai mari

În Java **orice element este un obiect**, în afară de datele primare.

Din Java **lipsește funcțiile și variabilele globale**, dar există metodele și variabilele statice ale claselor.

Clasele abstracte, neinstantiabile, servesc în general pentru definirea unor proprietăți sau operații comune ale mai multor clase și pentru a putea generaliza operațiile referitoare la acestea.

Dacă o clasă de obiecte are cel puțin o metodă abstractă, ea devine în întregime o clasă abstractă și nu poate fi instanțiată, adică nu se pot crea instanțe ale unei clase de obiecte abstracte.

Altfel spus, o clasă abstractă de obiecte este o clasă pentru care nu s-au precizat suficient de clar toate metodele astfel încât să poată fi folosită în mod direct.

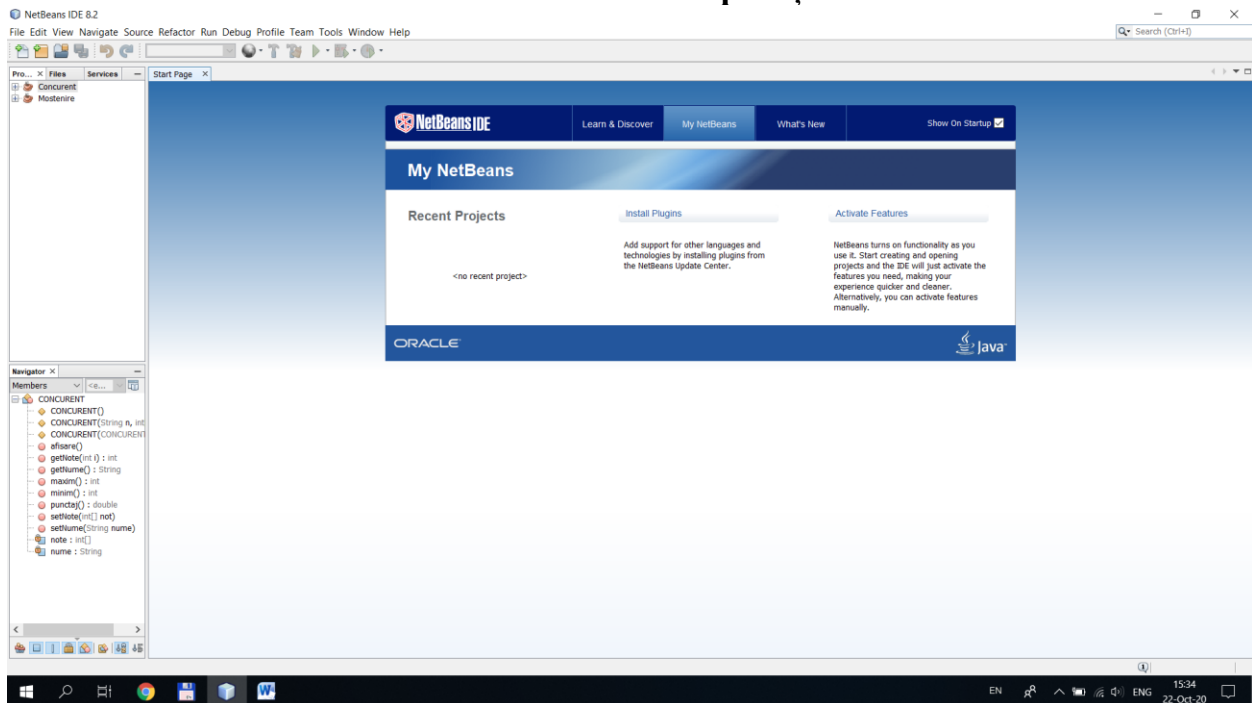
Derivarea claselor de obiecte

Operația prin care se definește o nouă clasă de obiecte pe baza uneia deja existente se numește **derivare**.

Reutilizarea codului se poate realiza prin două modalități:

- prin compunere: deci includerea de obiecte în cadrul altor obiecte
- prin extinderea clasei de bază: deci prin crearea unor obiecte pornind de la colecția de obiecte existente, completând-o, astfel pe cea din urmă

1.3. NetBeans IDE - mediul de dezvoltare al aplicațiilor Java



1.4. Aplicație propusă

Să se implementeze un proiect care definește clasa *farmacie* având ca date membre: denumirea medicamentului, prețul medicamentului și cantitatea.

De asemenea să se implementeze o clasă de test (clasa principală care va fi lansată în execuție la rularea aplicației) care va conține *metoda main* și în care se vor instanția obiecte de tipul clasei *farmacie*, utilizând constructorii implementați (implicit, explicit și de copiere) și să se afișeze la consolă datele acestor obiecte.

Pentru a obține informațiile obiectelor instanțiate, respectiv pentru a modifica informațiile acestor obiecte se vor utiliza funcțiile de acces de tip *get* și de tip *set*.

Să se creeze o metodă Java care să determine valoarea totală pentru fiecare medicament (preț*cantitate).

Pentru implementarea proiectului se va utiliza mediul NetBeans IDE.